Section:	BCS-2G
	Project

Submission Instructions

Course:	Object Oriented Pro

(Please read the following instructions carefully <u>before submitting</u> the project).

1. Single .zip file named RollNumber.zip (e.g. 22L-1234.zip) + GitHub repository link. 2.All `.h` and `.cpp` source files + `README.md`
3. It should be clear that your project will not get any credit if:
   - The project is submitted after the due date.
   - Project is plagiarized from any source. (online/other students/etc.)

Important Notes:

I. Work on this assignment INDIVIDUALLY.
II. Submit ONE `.zip` named exactly as your roll number (e.g. 22L-1234.zip).
III. The zip must contain all `.h` and `.cpp` files and a `README.md` explaining how to compile and run.
IV. Each class must be declared in its own `.h` file and defined in its own `.cpp` file. V. You must also push your complete project to a public GitHub repository. Paste the repository link inside your `README.md`. Submissions without a valid GitHub link will lose 5 marks.
VI. `std::vector` is NOT allowed anywhere in the code.
VII. There should not be any memory leakage in your program, otherwise it will result in negative marks.
VIII. Make separate functions for input and output. Your program must be as modular as possible. Your main() should be a sequence of function calls only.
IX. You are not allowed to use global variables and go-to instruction.
X. All allocations of pointers should be dynamic. Use of static arrays is prohibited. XI. Remember to delete the allocated memory when it is no longer needed. XII. All the data will be given by the user. Do not submit a program that runs on hard coded inputs.
XIII. All data must be stored in and read from `.txt` files. No data is hardcoded anywhere in the code.
XIV. Pass the pointers to functions instead of subscript notation [].
XV. Make proper functions to solve the problems.
XVI. Use of `std::string` and all built-in functions like `strcmp()` and

`strtok()`, etc., is strictly prohibited.

XVII. Write readable, well-formatted and well-commented code.

Problem Statement

You are to design and implement a Hospital Management System for a fictional hospital called *MediCore*. The system supports three types of users: Patients, Doctors and an Admin. It manages appointments, prescriptions, billing and medical records. All data must survive program restarts, meaning every change is immediately written to the relevant file and on startup the system reads all files to restore the previous state.

System Structure

Below is the required class structure. You must implement all classes listed. You may add more classes if you need them but you may not remove any class listed here. The relationships between classes: which ones own others, which ones merely reference others and which ones inherit from others are for you to determine based on what you have learned in this course. Choosing the wrong relationship type will cost marks.

Operator Overloading Requirements

You are required to overload operators wherever it makes logical sense throughout the project. The following are the **minimum required** overloads.

- `==` on `Appointment` — to check if two appointments conflict (same doctor, same date, same time slot where neither is cancelled).
- `<<` on `Patient`, `Doctor` and `Appointment` — for formatted console output.
- `-=` on `Patient` — to deduct an amount from the patient's balance.
- `==` on `Patient` — to compare two patients by ID.
- `==` on `Doctor` — to compare two doctors by ID.

Think carefully about where else operators can replace verbose function calls and make the code cleaner.

User Roles & Menus

On Startup:

```
Welcome to MediCore Hospital Management System
==============================================
Login as:
1. Patient
2. Doctor
3. Admin
4. Exit
```

The user selects their role then enters their ID and password, validated against the corresponding file. After 3 consecutive failed login attempts, the session is locked, a message is
displayed (`Account locked. Contact admin.`) and the attempt is
logged to `security_log.txt`

Patient Menu

```
Welcome, [Patient Name]
Balance: PKR [balance]
========================
1. Book Appointment
2. Cancel Appointment
3. View My Appointments
4. View My Medical Records
5. View My Bills
6. Pay Bill
7. Top Up Balance
8. Logout
```

1. Book Appointment

- Prompt: `Enter specialization to search (e.g. Cardiology):` • Display all doctors with that exact specialization. The comparison must be case insensitive, implemented manually using `tolower` on each character — do not use any library function for this. Show doctor ID, name and fee. If no doctors are found print `No doctors available for that specialization.` and return to menu. • Prompt: `Enter Doctor ID:` — validate it exists in the loaded doctors. If invalid print `Doctor not found.` and return to menu.
- Prompt: Enter date (DD-MM-YYYY): — validate strictly: day 01–31, month 01–12, year must be the current year or later. If invalid print `Invalid date. Use format DD MM-YYYY.` and re-prompt. After 3 failed attempts return to menu.
- Display available time slots for that doctor on that date. The 8 fixed daily slots

are: `09:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00.` A slot is unavailable if an appointment already exists for the same doctor ID, same date and same time slot where that appointment's status is not `cancelled`.

- Prompt: `Enter time slot (e.g. 09:00):` — validate it is one of the 8 valid slots. If the slot is taken throw `SlotUnavailableException`, catch it, print the exception message and re-display available slots.
- Check patient balance `>=` doctor fee using the overloaded `==` or comparison logic on the Patient object. If insufficient throw `InsufficientFundsException`, catch it, print the exception message and return to menu.
- Deduct the fee from the patient's balance using the overloaded `-=` operator. Generate a new appointment ID by reading `appointments.txt`, finding the current maximum ID and adding 1. Write the new appointment to `appointments.txt` with status `pending`. Write a corresponding bill to `bills.txt` with status `unpaid`. Update the patient's balance in patients.txt. Print: `Appointment booked successfully. Appointment ID: [id].`

2. Cancel Appointment

- Display all `pending` appointments for this patient: Appointment ID | Doctor Name | Date | Time Slot.
- If none print `You have no pending appointments.` and return to menu. • Prompt: `Enter Appointment ID to cancel:` — validate it belongs to this patient and has status `pending`. If not print `Invalid appointment ID.` and return to menu.
- Update the appointment's status to `cancelled` in `appointments.txt`. Refund the doctor's fee to the patient's balance using the overloaded `+=` operator. Update the patient record in `patients.txt`. Update the corresponding bill's status to `cancelled` in `bills.txt`. Print: `Appointment cancelled. PKR [fee] refunded to your balance.`

3. View My Appointments

- Display all appointments for this patient: ID | Doctor Name | Specialization | Date | Time Slot | Status.
- Sort by date ascending. Implement the sort yourself — do not use any library sort function.
- If none print `No appointments found.`

4. View My Medical Records

- Display all prescriptions for this patient grouped by appointment: Date | Doctor Name | Medicines | Notes.
- Sort by date descending (most recent first). Implement the sort yourself. • If none print `No medical records found.`

5. View My Bills

- Display all bills for this patient: Bill ID | Appointment ID | Amount (PKR) | Status |

Date. • Show the total outstanding unpaid amount at the bottom.
• If none print `No bills found.`

6. Pay Bill

- Display all `unpaid` bills for this patient.
- If none print `No unpaid bills.` and return to menu.
- Prompt: `Enter Bill ID to pay:` — validate it belongs to this patient and has status `unpaid`.
- Check balance >= bill amount. If not throw `InsufficientFundsException`, catch it, print the exception message and return to menu.
- Deduct the amount from balance using the overloaded `-=` operator. Update bill status to `paid` in `bills.txt`. Update patient balance in `patients.txt`. Print: `Bill paid successfully. Remaining balance: PKR [balance].`

7. Top Up Balance

- Prompt: `Enter amount to add (PKR):` — validate it is a positive number greater than 0. If invalid throw `InvalidInputException`, catch it, print the exception message and re-prompt. Allow up to 3 attempts before returning to menu.
- Add the amount to the patient's balance using the overloaded `+=` operator. Update `patients.txt`. Print: `Balance updated. New balance: PKR [balance].`

Doctor Menu

```
Welcome, Dr. [Name] | Specialization: [spec]
================================================
1. View Today's Appointments
2. Mark Appointment Complete
3. Mark Appointment No-Show
4. Write Prescription
5. View Patient Medical History
6. Logout
```

1. View Today's Appointments

- Get today's date using `time()` and `strftime()` from `<ctime>`. Display all appointments where the doctor ID matches and the date matches today: Appointment ID | Patient Name | Time Slot | Status. Sort by time slot ascending. If none print `No appointments scheduled for today.`

2. Mark Appointment Complete

- Display today's `pending` appointments for this doctor.
- Prompt: `Enter Appointment ID:` — validate it belongs to this doctor, has status `pending` and is dated today.

- Update status to `completed` in `appointments.txt`. Print: `Appointment marked as completed.`

3. Mark Appointment No-Show

- Same as above but updates the appointment status to `no show` in `appointments.txt` and updates the corresponding bill's status to `cancelled` in `bills.txt`. No refund is issued. Print: `Appointment marked as no-show.`

4. Write Prescription

- Prompt: `Enter Appointment ID:` — validate it belongs to this doctor and has status `completed`. Then check `prescriptions.txt` — if a prescription already exists for this appointment ID print `Prescription already written for this appointment.` and return to menu.
- Prompt: `Enter medicines (format: MedicineName Dosage; e.g. Paracetamol 500mg;Amoxicillin 250mg):` — read into a `char medicines[500]`. If the input exceeds 499 characters truncate it silently.
- Prompt: `Enter notes (max 300 chars):` — read into `char notes[300]`. Truncate silently if longer.
- Generate a new prescription ID using the same method as appointment ID generation. Write to `prescriptions.txt`. Print: `Prescription saved.`

5. View Patient Medical History

- Prompt: `Enter Patient ID:` — validate the patient exists AND has at least one `completed` appointment with this doctor. If not print `Access denied. You can only view records of your own patients.` and return to menu.
- Display all prescriptions for that patient written by this doctor sorted by date descending.

Admin Menu

```
Admin Panel — MediCore
======================
1. Add Doctor
2. Remove Doctor
3. View All Patients
4. View All Doctors
5. View All Appointments
6. View Unpaid Bills
7. Discharge Patient
8. View Security Log
9. Generate Daily Report
10. Logout
```

1. Add Doctor

- Prompt for: name (max 50 chars), specialization (max 50 chars), contact (must be exactly 11 digits all numeric — validate with `Validator`), password (minimum 6 characters — validate with `Validator`), consultation fee (must be a positive float — validate with `Validator`).
- Generate a new doctor ID (max existing ID + 1). Write the new record to `doctors.txt`. Print: `Doctor added successfully. ID: [id].`

2. Remove Doctor

- Display all doctors: ID | Name | Specialization | Fee.
- Prompt: `Enter Doctor ID to remove:`
- Check if this doctor has any appointments with status `pending` in `appointments.txt`. If yes print `Cannot remove doctor with pending appointments. Cancel or reassign them first.` and return to menu.
- Delete the doctor's record from `doctors.txt` using `FileHandler`. Print: `Doctor removed.`

3. View All Patients

- Display: ID | Name | Age | Gender | Contact | Balance | Count of unpaid bills.

4. View All Doctors

- Display: ID | Name | Specialization | Contact | Fee.

5. View All Appointments

- Display all appointments: ID | Patient Name | Doctor Name | Date | Time Slot | Status. Sorted by date descending.

6. View Unpaid Bills

- Display all `unpaid` bills across all patients: Bill ID | Patient Name | Amount (PKR) | Date.
- For any bill whose date is more than 7 days before today (calculated using `difftime`() on parsed date fields) append `[OVERDUE]` to the date column.

7. Discharge Patient

- Prompt: `Enter Patient ID:`
- If the patient has any bills with status `unpaid` print `Cannot discharge patient with unpaid bills.` and return to menu.
- If the patient has any appointments with status `pending` print `Cannot discharge patient with pending appointments.` and return to menu.
- Copy the patient's full record (their row from `patients.txt` along with all their rows from `appointments.txt`, `prescriptions.txt` and `bills.txt`)

into discharged.txt using the same comma-separated format. Then delete their records from each of those four files using `FileHandler`. Print: `Patient discharged and archived successfully.`

8. View Security Log

- Read and display the contents of `security_log.txt` line by line. If empty print `No security events logged.`

9. Generate Daily Report

- Get today's date. Display the following, all derived by reading existing files — no separate report file is created:
  - `Total appointments today: [n] (Pending: [n] Completed: [n] No-show: [n] Cancelled: [n])`
  - `Revenue collected today (paid bills): PKR [amount]`
  - Patients with outstanding unpaid bills: Patient Name | Total Owed
  - Doctor-wise summary for today: Doctor Name | Completed | Pending | No-show

File Formats

All files use comma-separated values, one record per line with no spaces around commas.

`patients.txt`

```
patient_id,name,age,gender,contact,password,balance
1,Ahmed Ali,28,M,03001234567,pass123,5000.00
```

`doctors.txt`

```
doctor_id,name,specialization,contact,password,fee
1,Dr. Sara Khan,Cardiology,03111234567,doc456,1500.00
```

`admin.txt`

```
admin_id,name,password
1,Admin,admin123
```

`appointments.txt`

```
appointment_id,patient_id,doctor_id,date,time_slot,st
atus 1,1,1,15-04-2025,09:00,completed
```

`prescriptions.txt`

```
prescription_id,appointment_id,patient_id,doctor_id,date,medicine
s,no tes
1,1,1,1,15-04-2025,Paracetamol 500mg;Amoxicillin 250mg,Take
after  meals
```

`bills.txt`

```
bill_id,patient_id,appointment_id,amount,status,date
1,1,1,1500.00,paid,15-04-2025
```

`security_log.txt`

```
timestamp,role,entered_id,result
15-04-2025 09:32:11,Patient,99,FAILED
```

`discharged.txt` Same format as `patients.txt`. One discharged patient record per line.

Required Classes

You must implement all of the following classes. The choice of attributes, methods, constructors and relationships is yours — that is part of what is being assessed.

- **Person** — Abstract base class. Holds common identity attributes shared by all user types. Must have at least two pure virtual methods. `Patient`, `Doctor` and `Admin` inherit from it.
- **Patient** — Represents a registered patient. Must overload `+=` to add to balance, `-=` to deduct from balance, `==` to compare by ID and `<<` for display.
- **Doctor** — Represents a hospital doctor with a specialization and consultation fee. Must overload `==` to compare by ID and `<<` for display.
- **Admin** — Represents the system administrator. Loaded from `admin.txt` on startup. • **Appointment** — Represents a single appointment. Must overload `==` to detect a scheduling conflict (same doctor, same date, same time slot where neither is cancelled). Must overload `<<` for display.
- **Bill** — Represents a charge generated from an appointment.
- **Prescription** — Represents medicines and notes written by a doctor for a completed appointment.
- **Storage<T>** — A generic template class. Internally uses a static array `T data[100]` and an `int count`. Must provide at minimum: add, remove by ID, find by ID, get all and size. No vectors inside.

- **FileHandler** — The only class permitted to perform file I/O. Must support loading all entity types from their files into `Storage<T>` or arrays, appending new records, updating a record by ID and deleting a record by ID.
- **Validator** — The only class permitted to contain input validation logic. Must validate: IDs, dates in DD-MM-YYYY format, time slots, contact numbers, passwords, positive floats and menu choices.
- **HospitalException** — Base class for all custom exceptions. Stores a `char message[200]` and has a virtual w`hat()` method returning the message. • **FileNotFoundException** — Thrown when a required file cannot be opened on startup. • **InsufficientFundsException** — Thrown when a patient's balance is less than the required amount.
- **InvalidInputException** — Thrown when user input fails validation.
- **SlotUnavailableException** — Thrown when a patient attempts to book an already occupied time slot.